

COMPUTER SCIENCE TRIPOS Part IB – 2023 – Paper 4

2 Compiler Construction (jdy22)

Here is an OCaml definition of the Ackermann function, `ack`:

```
let rec ack m n =
  if m = 0 then n+1
  else if n = 0 then ack (m-1) 1
  else ack (m-1) (ack m (n-1))
```

- (a) You would like to run `ack` on an old system with limited stack space and no support for closures, and decide to rewrite it in stages.

- (i) Rewrite the `ack` function to produce a function `ack_cps` in continuation-passing style so that the function

```
let ack_1 m n = ack_cps m n (fun x -> x)
```

produces the same results as the function `ack`.

[5 marks]

Answer:

```
let rec ack_cps m n k =
  if m = 0 then k (n+1) else
  if n = 0 then ack_cps (m-1) 1 k else
  ack_cps m (n-1) (fun x -> ack_cps (m-1) x k)

let ack_1 m n = ack_cps m n (fun x -> x)
```

- (ii) Eliminate higher-order functions from your answer to Part (a)(i) by rewriting `ack_cps` as a function `ack_cps_dfn` in *defunctionalized* form so that the function

```
let ack_2 m n = ack_cps_dfn m n ID
```

produces the same results as the function `ack`.

[5 marks]

Answer:

```
type cont = ID | Cont of int * cont

let rec apply_cont : cont * int -> int = function
  | ID, x -> x
  | Cont (m, k), x -> ack_cps_dfn (m-1) x k
and ack_cps_dfn m n k =
  if m = 0 then apply_cont (k,n+1) else
  if n = 0 then ack_cps_dfn (m-1) 1 k else
  ack_cps_dfn m (n-1) (Cont (m, k))

let ack_2 m n = ack_cps_dfn m n ID
```

continuation-
passing
style

defunction-
alization

- (iii) Convert your answer from Part (a)(ii) to a function `ack_cps_dfn_list` that uses standard lists rather than custom data types [5 marks]

Answer:

```
let rec apply_cont2 = function
  | [], x -> x
  | m :: ms, x -> ack_cps_dfn_list (m-1) x ms
and ack_cps_dfn_list m n k =
  if m = 0 then apply_cont2 (k,n+1) else
  if n = 0 then ack_cps_dfn_list (m-1) 1 k else
  ack_cps_dfn_list m (n-1) (m :: k)

let ack_3 m n = ack_cps_dfn_list m n []
```

stack, heap

- (b) Briefly comment on the way that `ack` and the transformed implementations in Parts (a)(i), (a)(ii) and (a)(iii) use memory, making reference to the stack and heap. [5 marks]

Answer: Each of the functions uses a similar amount of memory. The original `ack` function allocates memory on the stack, while each of the other implementations allocates on the heap. The memory usage of the transformed functions may be higher at a given point in the program's execution, because the allocated heap values will not be collected immediately.
